Far-Shot Velocity Tuning Guide

Profile: **HOOD_0_70** · Hood = 0.7 · 18 points, 70–155 in · Print & fill the right column on the field

#	Dist (in)	Hood	Start RPM	Tuned RPM
1	70	0.7	3180	
2	75	0.7	3200	
3	80	0.7	3230	
4	85	0.7	3280	
5	90	0.7	3330	
6	95	0.7	3380	
7	100	0.7	3440	
8	105	0.7	3530	
9	110	0.7	3610	

#	Dist (in)	Hood	Start RPM	Tuned RPM
10	115	0.7	3690	
11	120	0.7	3780	
12	125	0.7	3860	
13	130	0.7	3940	
14	135	0.7	**4030***	
15	140	0.7	**4110***	
16	145	0.7	**4200***	
17	150	0.7	**4280***	
18	155	0.7	**4370***	

Start RPMs come from the current curve, which **overshoots far** → expect to dial most **down**, especially 120+. ***Rows 14–18 are above the 4100 RPM cap** — spin the flywheel up empty first; whatever it tops out at is your real max range. Drop any row past the distance you can actually reach.

Setup (once)

1. **Put a fresh, full battery in.** RPM depends on voltage — tune near match voltage or the numbers won't hold up.
2. Deploy current code from Android Studio (Run).
3. On the Driver Station, select the **"Shooter Tuner"** OpMode (don't start it yet).
4. Connect a laptop to the robot's WiFi, open **FTC Dashboard** → `192.168.43.1:8080/dash`.
5. Set `hoodPosition = 0.7`, `targetRPM` = first row's value, `feedPower = 0`.
6. (Optional) Add **Target RPM** and **Actual RPM** to the Dashboard graph so you can see the flywheel settle.

For each row, top to bottom

7. **Tape-measure** the robot to the distance (goal → turret pivot). Place it there, turret aimed at the goal.
8. **Init + Start** the OpMode. Flywheel spins up.
9. **Wait for Actual RPM to settle** around target (bang-bang oscillates slightly — that's normal; wait until it hovers, not still climbing).
10. **Fire one ball** — flick `feedPower` to fire then back to 0 (or feed with intake). Watch where it lands.
11. **Adjust** `targetRPM`: lands **short** → +25–50; **overshoots** → –25–50.
12. **Re-fire**, letting RPM recover to target between shots.
13. When **3 shots in a row score**, write that RPM in the table. Stop the OpMode.
14. Move to the next distance, repeat.

Enter into code

15. Replace the `HOOD_0_70` table in `ShooterAimingModel.java` with your 18 pairs (ascending), and set the profile max to cover your top distance:

```
new Profile("HOOD_0_70", 0.7, 70.0, 155.0, new double[][] {
  {70.0, ___}, {75.0, ___}, {80.0, ___}, {85.0, ___}, {90.0, ___}, {95.0, ___},
  {100.0, ___}, {105.0, ___}, {110.0, ___}, {115.0, ___}, {120.0, ___}, {125.0, ___},
  {130.0, ___}, {135.0, ___}, {140.0, ___}, {145.0, ___}, {150.0, ___}, {155.0, ___},
});
```

16. Deploy, then **verify**: drive to a few random far spots and confirm it scores (the model interpolates between points).
17. **Commit** ("far shot table retuned, 18 pts").

Keep it honest

18. Same ball condition and battery state throughout — a tired battery mid-session skews the far numbers worst.
19. Quick check once: at a parked spot, the `Standby Dist (in)` telemetry should match your tape within an inch or two (match aims off odometry; you measure with tape).